

Image Recognition



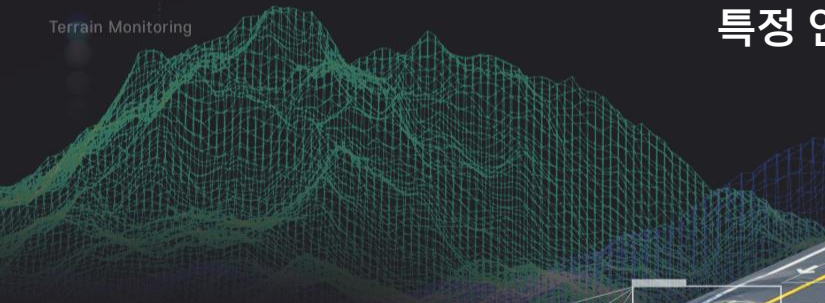
강원혁신플랫폼

컴퓨터 비전



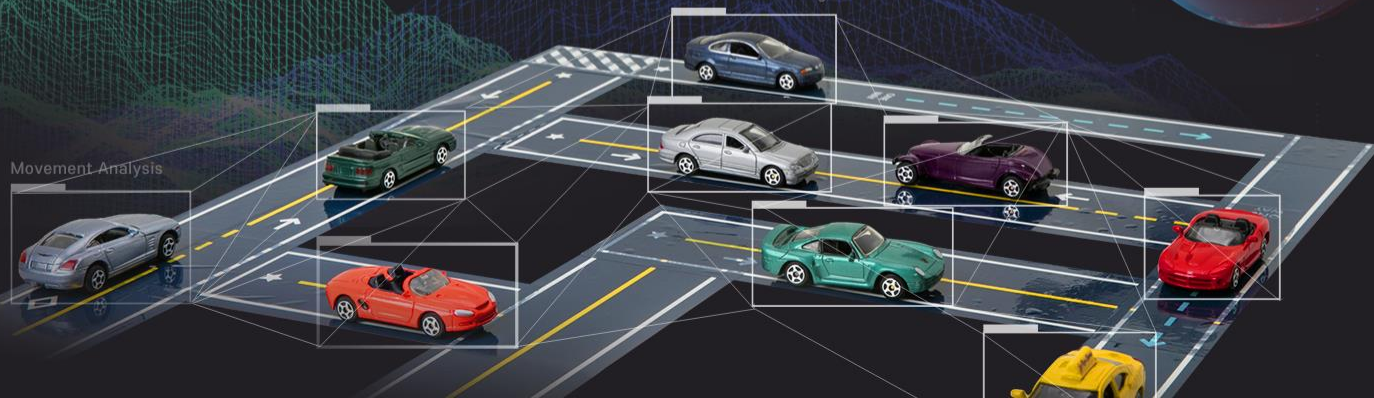
특정 인물 얼굴 인식(이미지)

Terrain Monitoring



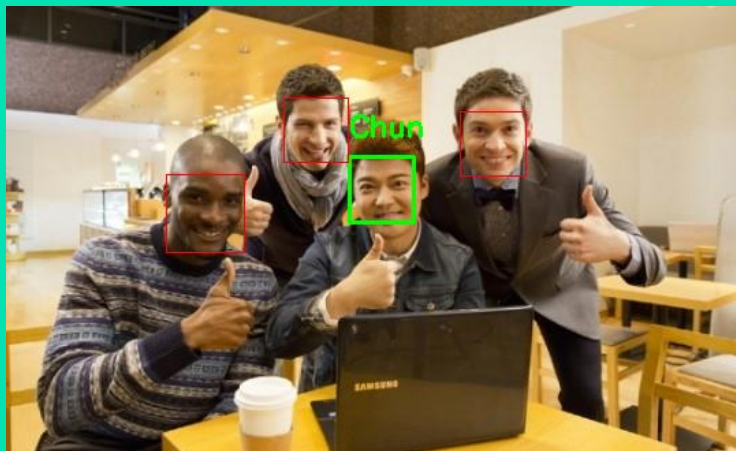
Autonomous Driving

Movement Analysis



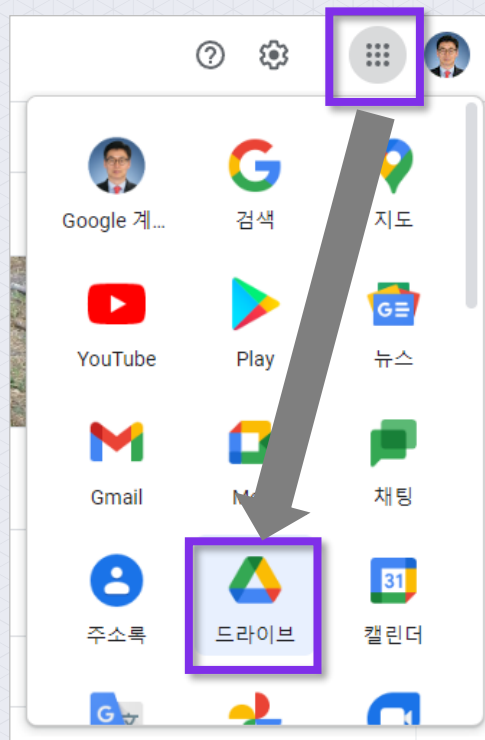
- 이미지에서 얼굴을 검출(detection)하고 얼굴 영역의 특정값과 우리가 미리 학습시켜 둔 'encodings.pickle' 파일의 특성값을 비교해서 이미지에 특정한 인물이 존재하는지 분류 예측하는 실습을 진행해보자

☞ 앞에서 전무현 방송인의 얼굴 이미지 사진에서 얼굴 특성값과 이름을 매칭하여 'encodings.pickle' 파일 이름으로 저장해 두었음

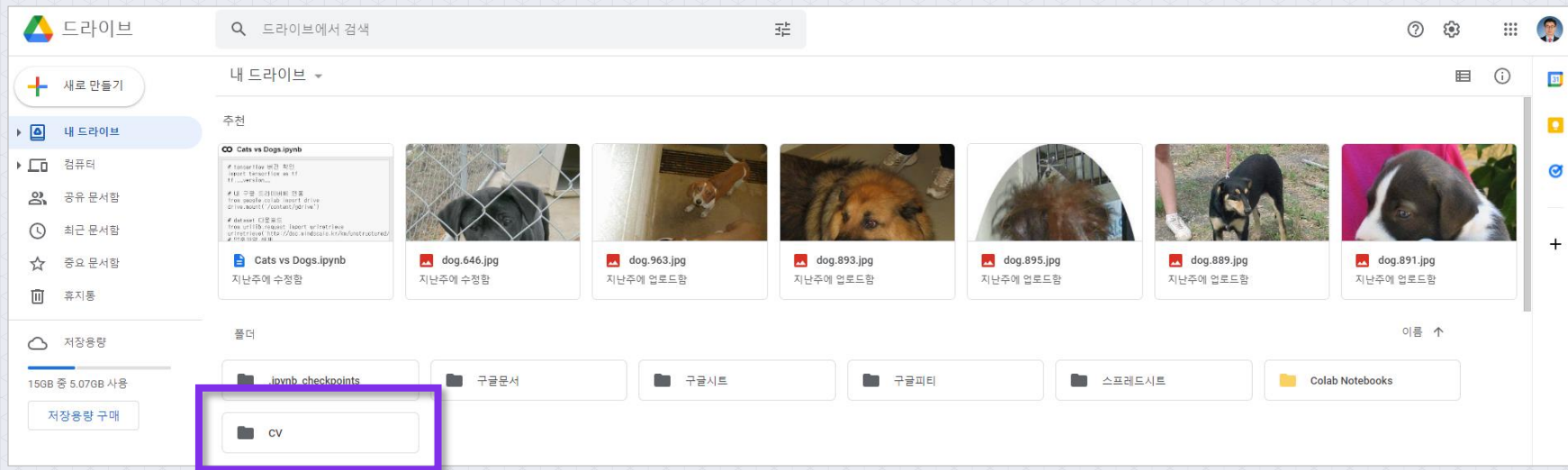


각자 본인의 사진을 준비하여
얼굴 특성값을 저장하여
실습해 보아도 좋을 것!

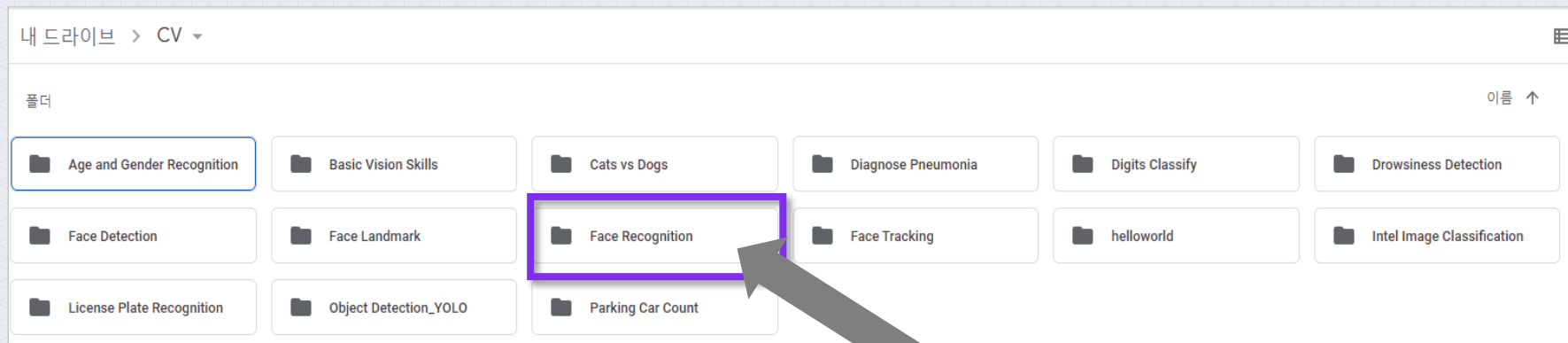
- 먼저, 구글 Colab을 이용해 보자
- 구글 크롬 브라우저에서 아래 그림과 같이 **구글 드라이브**를 클릭



구글 드라이브에서 CV 폴더를 클릭

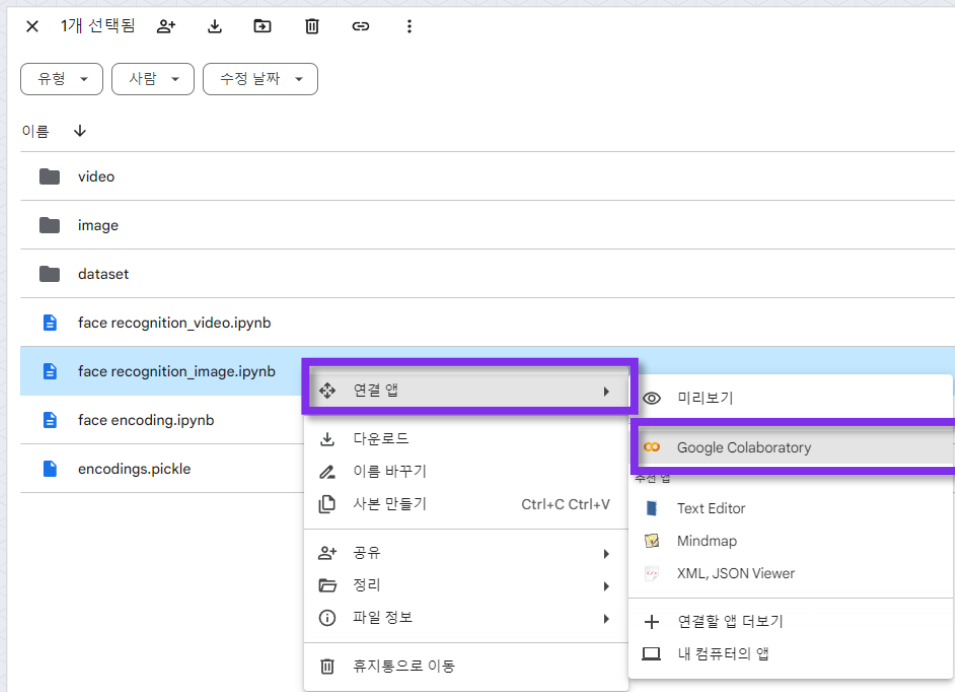


구글 드라이브 > CV > Face Recognition 폴더를 클릭



Face Recognition

- ❶ 아래의 그림의 image 폴더에서 face_recognition_image.ipynb를 선택 후 오른쪽 마우스를 클릭하여 “연결 앱 > Google Colaboratory”을 클릭



🔄 구글 Colab 노트북 화면이 보이면 실습 준비가 완료된 것임



face_recognition_image.ipynb ☆

파일 수정 보기 삽입 런타임 도구 도움말 모든 변경사항이 저장됨

+ 코드 + 텍스트

- face_recognition 은 안면을 인식하는 심플한 API for Python.
- face_recognition 패키지는 정확도면에서 한계가 있다.
- 실전에서는 face landmark를 활용한 이미지 전처리 과정(Alignment) 등을 통해 정확도를 높이고 있다.
- 동양인들 끼리의 얼굴 인식은 잘 안 된다.

✓ 21 초 # 내 구글 드라이브에 연동

```
from google.colab import drive
drive.mount('/content/gdrive')
```

Mounted at /content/gdrive

- ② 내 구글 드라이브에 연동
- ② 구글 Colab 노트북 화면에서 실행 버튼 클릭



- ② [Google Drive에 연결] > [계정 선택] > [구글 드라이브 접근 허용] 메뉴를 선택하면 gdrive에 연동
- ② gdrive에 연동되면 구글 드라이브에 저장된 데이터를 활용할 수 있음

구글 Colab 화면에서 내 구글 드라이브 연동

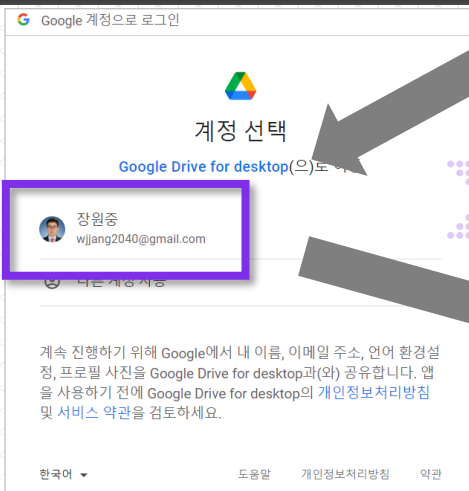
- [Google Drive에 연결] > [계정 선택] > [구글 드라이버 접근 허용]
- 아래 화면을 참고하여 **gdrive**에 연동

노트북에서 Google Drive 파일에 액세스하도록 허용하시겠습니까?

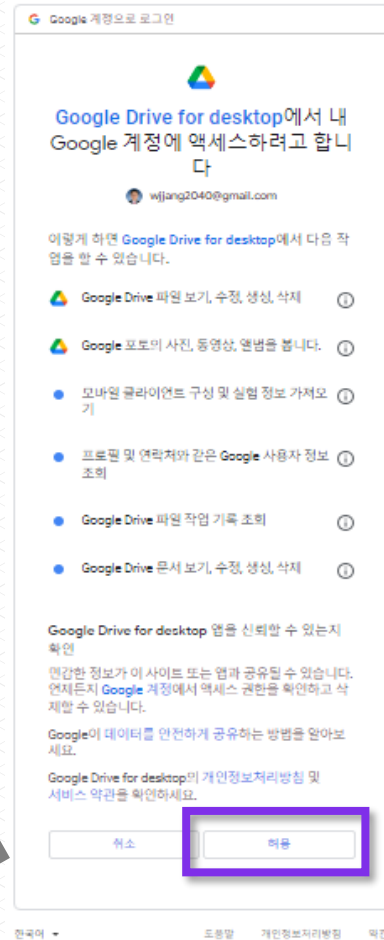
이 노트북에서 Google Drive 파일에 대한 액세스를 요청합니다. Google Drive에 대한 액세스 권한을 부여하면 노트북에서 실행되는 코드가 Google Drive의 파일을 수정할 수 있게 됩니다. 이 액세스를 허용하기 전에 노트북 코드를 검토하시기 바랍니다.

아니오

Google Drive에 연결



구글 드라이버 접근 허용



■ face_recognition 패키지의 상세정보 버전을 확인해 보자

- 🔄 확인 결과 'face_recognition' 패키지가 설치되지 않은 것을 볼 수 있음

```
1 # 'face_recognition' 모듈(라이브러리, 패키지)의 상세정보 버전 확인
2
3 !pip show face_recognition
```

```
⚠ WARNING: Package(s) not found: face_recognition
```

■ face_recognition 패키지가 잘 설치된 것을 확인할 수 있음

```
1 # face_recognition 패키지는 구글 Colab에서 기본으로 지원하지 않아 반드시 설치해야 한다.  
2  
3 !pip3 install face_recognition          # face_recognition install  
4  
5 # !pip3 uninstall face_recognition      # face_recognition uninstall
```

```
Collecting face_recognition  
  Downloading face_recognition-1.3.0-py2.py3-none-any.whl (15 kB)  
Collecting face-recognition-models>=0.3.0 (from face_recognition)  
  Downloading face_recognition_models-0.3.0.tar.gz (100.1 MB)  
----- 100.1/100.1 MB 11.7 MB/s eta 0:00:00  
Preparing metadata (setup.py) ... done  
Requirement already satisfied: Click>=6.0 in /usr/local/lib/python3.10/dist-packages (from face_recognition) (8.1.6)  
Requirement already satisfied: dlib>=19.7 in /usr/local/lib/python3.10/dist-packages (from face_recognition) (19.24.2)  
Requirement already satisfied: numpy in /usr/local/lib/python3.10/dist-packages (from face_recognition) (1.22.4)  
Requirement already satisfied: Pillow in /usr/local/lib/python3.10/dist-packages (from face_recognition) (9.4.0)  
Building wheels for collected packages: face-recognition-models  
  Building wheel for face-recognition-models (setup.py) ... done  
  Created wheel for face-recognition-models: filename=face_recognition_models-0.3.0-py2.py3-none-any.whl size=100566169  
  Stored in directory: /root/.cache/pip/wheels/7a/eb/cf/e9ced74122b679557f597bb7c8e4c739cfac526db1fd523d  
Successfully built face-recognition-models  
Installing collected packages: face-recognition-models, face_recognition  
Successfully installed face-recognition-models-0.3.0 face_recognition-1.3.0
```

- 필요한 패키지와 모듈을 불러옴
- 구글 Colab 노트북 화면에서 실행 버튼 클릭

③ # 필요한 패키지와 모듈을 불러 온다.

```
import cv2
from google.colab.patches import cv2_imshow
import face_recognition
import pickle
import time
import dlib
```

```
print('cv2 version :', cv2.__version__)
print('dlib version :', dlib.__version__)
```

```
cv2 version : 4.6.0
dlib version : 19.24.0
```

원본 이미지와 얼굴 특성값 파일 위치, 이미지에서 주요 특징을 캡처하는 표현 방법을 “cnn”으로 설정

주요 특징 캡처하는 시간
: 조금 느림

정확도 : 높음

```

1  # 원본 이미지와 얼굴 특성값 저장 위치 설정
2  # - 이미지에서 주요 특징을 캡처하는 표현 방법 : cnn
3  # - cnn : 조금 느리지만 정확도가 높음
4
5  image_file1 = 'gdrive/My Drive/CV/Face Recognition/image/meeting1.jpg' # 원본 이미지
6  image_file2 = 'gdrive/My Drive/CV/Face Recognition/image/meeting2.jpg' # 원본 이미지
7
8  encoding_file = 'gdrive/My Drive/CV/Face Recognition/encodings.pickle' # 저장해 두었던 특성 파일
9  unknown_name = 'Unknown' # 이름을 모를 경우 Unknown 으로 지정
10
11 # Either cnn or hog. The CNN method is more accurate but slower. HOG is faster but less accurate.
12 model_method = 'cnn'
    
```

■ 원본 이미지(1)을 읽어와서 출력

- 🔄 원본 이미지 형상 : (354, 536, 3)
(형상은 세로(H), 가로(W), 채널(C) 순)

```
1 # 원본 이미지(1)를 읽어 온다.  
2 image1 = cv2.imread(image_file1)  
3  
4 print("image shape:", image1.shape) # image shape: (354, 536, 3)  
5 print("### Original Image ###")  
6 cv2.imshow(image1)
```



■ 원본 이미지(2)을 읽어와서 출력

🔄 원본 이미지 형상 : (328, 540, 3)
(형상은 세로(H), 가로(W), 채널(C) 순)

```
1 # 원본 이미지(2)를 읽어 온다.  
2 image2 = cv2.imread(image_file2)  
3  
4 print("image shape:", image2.shape) # image shape: (328, 540, 3)  
5 print("### Original Image ###")  
6 cv2_imshow(image2)
```



■ 저장해 두었던 얼굴 특성값(encodings) 파일을 불러옴

🔄 전현무 방송인의 얼굴의 특성값 : 128 크기의 벡터로 계산되어 저장된 것을 알 수 있음

```
1 # 저장해 두었던 encodings(특성값) 파일을 불러온다.
2 data = pickle.loads(open(encoding_file, "rb").read())
3
4 print("type:", type(data))    # type: <class 'dict'>
5 print("keys", data.keys())    # keys dict_keys(['encodings', 'names'])
6 print("values:", len(data.get('encodings')[0]), data.get('encodings')[0])
7 print("values:", data.get('names'))
```

values: 128 [-0.09229942 0.03761727 0.03389517 -0.01408569 -0.11893666 -0.04296163
-0.02862475 -0.09126651 0.1302492 -0.136141 0.16108797 -0.04563347
-0.20622247 -0.08739801 -0.085503 0.15205184 -0.15774134 -0.13390714
-0.05035019 -0.00143872 0.09135978 0.03835174 0.01379618 -0.05379851
-0.05704164 -0.38159704 -0.10577583 -0.05868653 -0.01996204 -0.04913521
-0.03498504 0.10400848 -0.17031103 -0.08585772 0.00770638 0.09675372
0.01378377 -0.0424266 0.16537699 0.01374661 -0.25501731 -0.01648122
0.05126925 0.24238919 0.1358026 0.02915912 -0.0200779 -0.13080323
0.1001433 -0.17419206 0.03079278 0.11761235 0.09813562 0.0302905
0.01498112 -0.04760741 0.10297292 0.09403937 -0.19751999 -0.04012734
0.14391324 -0.10793913 -0.05710856 -0.05150723 0.17628026 0.07252432
-0.03975128 -0.17408653 0.12144931 -0.1764311 -0.09191512 0.0606965
-0.0804313 -0.17453927 -0.29059491 -0.01253504 0.37180018 0.14620435
-0.16211545 0.03877465 -0.03036392 -0.0143353 0.21439359 0.10431002
-0.0573099 0.08544192 -0.12094741 0.00546108 0.20904131 -0.02302563
-0.06869844 0.18634835 0.03773041 0.06341273 0.08954532 0.00686301
-0.01243788 0.03873602 -0.1836317 -0.01022412 0.10953911 -0.06280994
-0.04130162 0.09070587 -0.12510082 0.10474179 -0.03918144 0.00527665
0.03064633 -0.11650962 -0.09388351 -0.02239166 0.13738495 -0.18850376
0.11356711 0.14697406 0.01392272 0.13400608 0.13681641 0.06062232
-0.05418234 -0.1001198 -0.27050662 -0.0422022 0.14997393 -0.01974767
0.11343588 -0.01601369]

values: ['Chun', 'Chun', 'Chun', 'Chun', 'Chun', 'Chun', 'Chun', 'Chun', 'Chun', 'Chun']

```
# 얼굴 인식 및 출력하는 함수 정의
def detectAndDisplay(image):
    start_time = time.time() # 시작시간
    # 이미지를 face_recognition 에서 처리하기 위해 RGB 형식으로 변경
    rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

    # 얼굴을 detection 하고 boxing 한다.
    boxes = face_recognition.face_locations(rgb, model=model_method)

    # 얼굴 영역(boxing Area)만 encoding(특성값)을 계산한다.
    encodings = face_recognition.face_encodings(rgb, boxes)

    # detection 된 얼굴의 이름을 저장할 배열
    names = []

    for encoding in encodings:
        # boxing 된 특성이 pickle 파일의 encodings(특성값) 과 매칭되는 것이 있는지 찾아낸다.
        matches = face_recognition.compare_faces(data["encodings"], encoding)
        print(matches)
        name = unknown_name # 'Unknown'으로 초기화

    # check to see if we have found a match
    if True in matches:
        matchedIdxs = [i for (i, b) in enumerate(matches) if b] # 매칭된 pickle 파일의 True 인덱스 번호를 찾아온다.
        counts = {} # 매칭된 얼굴별로 이름과 갯수 Count 수를 Dictionary 타입으로 저장한다.

        for i in matchedIdxs:
            name = data["names"][i] # 매칭된 True 인덱스 번호의 "names" value 를 찾아온다.
            counts[name] = counts.get(name, 0) + 1 # counts dict의 "name" 키값의 value 에 count 누적

        name = max(counts, key=counts.get) # counts dict에서 key값(이름)들 중에서 가장 value 값(카운팅)이 큰
        # names 배열에 이름을 추가
        names.append(name)

    # dection 된 얼굴의 boxing 위치 좌표와 이름 매칭
    for ((top, right, bottom, left), name) in zip(boxes, names):
        color = (0, 255, 0) # boxing color
        line = 2 # boxing line 두께
        if(name == unknown_name): # 만약 unknown 이라면
            color = (0, 0, 255)
            line = 1
            name = '' # name 표시 안함

        cv2.rectangle(image, (left, top), (right, bottom), color, line)
        y = top - 15 if top - 15 > 15 else top + 15
        cv2.putText(image, name, (left, y), cv2.FONT_HERSHEY_SIMPLEX, 0.7, color, line)

    end_time = time.time()
    process_time = end_time - start_time
    print("=== A frame took {:.3f} seconds".format(process_time))
    # show the output image
    cv2.imshow(image)
```

■ 원본 이미지(1)에서 얼굴 인식 후 전현무 방송인 얼굴이 존재하는지 예측함

- 🔄 이미지 속 여러 명의 사람들 중에서 특정한 인물(여기서는 전현무 방송인)을 찾아 잘 표시해 주고 있음을 확인

```
1 # 원본 이미지(1)로 얼굴 검출 후 전현무 얼굴이 존재하는지 예측  
2 detectAndDisplay(image1)
```

```
➡ [True, True, True, True, True, True, True, True, True, True]  
[False, False, False, False, False, False, False, False, False, False, False]  
[False, False, False, False, False, False, False, False, False, False, False]  
[False, False, False, False, False, False, False, False, False, False, False]  
=== A frame took 8.106 seconds
```

실행 시간은 8.106 초가 소요



model=cnn

■ 원본 이미지(2)에서 얼굴 인식 후 전현무 방송인 얼굴이 존재하는지 예측함

- 🔄 이미지 속 여러 명의 사람들 중에서 특정한 인물(여기서는 전현무 방송인)을 찾아 잘 표시해 주고 있음을 확인

```
1 # 원본 이미지(2)로 얼굴 검출 후 전무현 얼굴이 존재하는지 예측  
2 detectAndDisplay(image2)
```

```
➡ [False, False, False, False, False, False, False, False, False, False]  
[True, True, True, True, True, True, True, True, True, True]  
[False, False, False, False, False, False, False, False, False, False]  
[False, False, False, False, False, False, False, False, False, False]  
=== A frame took 0.096 seconds
```

실행 시간은 0.096 초가 소요



model=cnn

원본 이미지와 얼굴 특성값 파일 위치, 이미지에서 주요 특징을 캡처하는
표현 방법을 “hog”으로 설정

주요 특징 캡처하는 시간
: 조금 빠름

정확도 : 낮음

```
1 # 원본 이미지와 얼굴 특성값 저장 위치 설정
2 # - 이미지에서 주요 특징을 캡처하는 표현 방법 : hog
3 # - hog : 조금 빠르지만 정확도가 낮음
4
5 image_file1 = 'gdrive/My Drive/CV/Face Recognition/image/meeting1.jpg' # 원본 이미지
6 image_file2 = 'gdrive/My Drive/CV/Face Recognition/image/meeting2.jpg' # 원본 이미지
7
8 encoding_file = 'gdrive/My Drive/CV/Face Recognition/encodings.pickle' # 저장해 두었던 특성 파일
9 unknown_name = 'Unknown' # 이름을 모를 경우 Unknown 으로 지정
10
11 # Either cnn or hog. The CNN method is more accurate but slower. HOG is faster but less accurate.
12 model_method = 'hog'
```


■ 원본 이미지(1)에서 얼굴 인식 후 전현무 방송인 얼굴이 존재하는지 예측함

🔄 이미지 속 여러 명의 사람들 중에서 특정한 인물(여기서는 전현무 방송인)을 찾아 표시해 주고 있지만 여러 번 표시된 것을 확인

```
1 # 원본 이미지(1)로 얼굴 검출 후 전현무 얼굴이 존재하는지 예측  
2 detectAndDisplay(image1)
```

```
➡ [True, True, True, True, True, True, True, True, True, True]  
[False, False, False, False, False, False, False, False, False, False]  
[False, False, False, False, False, False, False, False, False, False]  
=== A frame took 0.181 seconds
```

실행 시간은 0.181 초가 소요

model=hog



■ 원본 이미지(2)에서 얼굴 인식 후 전현무 방송인 얼굴이 존재하는지 예측함

🔄 이미지 속 여러 명의 사람들 중에서 특정한 인물(여기서는 전현무 방송인)을 찾아 표시해 주고 있지만 여러 번 표시된 것을 확인

```
1 # 원본 이미지(2)로 얼굴 검출 후 전무현 얼굴이 존재하는지 예측  
2 detectAndDisplay(image2)
```

```
➡ [True, True, True, True, True, True, True, True, True, True]  
[False, False, False, False, False, False, False, False, False, False]  
[False, False, False, False, False, False, False, False, False, False]  
=== A frame took 0.175 seconds
```

실행 시간은 0.175 초가 소요

model=hog

